



Hackathon Vibe Coding Handbook

Keep this open during the hackathon. Copy, paste, ship.

Tools I use

- [Claude Code](#): the AI coding agent doing the actual work.
- [Drawa](#) (free and open source): I built this because I kept losing track of what my agent was doing in the terminal. It puts your agent sessions on a canvas in your browser: see which files each one touched, review the diffs, comment on plans, and drop notes and sketches right next to the work. I keep a couple of sessions open side by side the whole hackathon.

```
curl -fsSL https://raw.githubusercontent.com/HimalayanNomads/drawa/main/install.sh  
| sh  
cd your-project && drawa
```

Runs **Claude Code**, **Codex** or **OpenCode** (free models, no key needed).

Most useful plugins

- **Caveman**: saves tokens by making the AI say things straight to the point.
- **Ponytail**: cuts off over-engineering. Less code, fewer bugs.
- **Impeccable**: a frontend design skill. Ask it to critique, polish or redesign a page and the UI stops looking AI-generated.
- **Brag**: turns your hackathon project into a launch video. Uses a lot of tokens, so run it at the end.

Install them from Claude Code with `/plugin`. Impeccable is a skill, not a plugin, so it installs a bit differently.

The workflow I usually follow

1. **Plan it out**. Ask for a plan first, no code yet.
2. **Get a Mermaid diagram** of the system architecture.
3. **Verify it**. Does it match what you want to build?
4. **Make changes** if needed.
5. **Send it back** to the AI.
6. **Iterate** until the architecture is what you want. Only then let it code.
7. **Let it check its own work**. End every coding prompt with "run it / build it and fix any errors before you tell me it's done."

Timebox the planning. Iterating on architecture is worth it, but at a hackathon give it ~30–45 minutes max. A good-enough plan you build beats a perfect one you don't.

I do this loop in Drawa's plan mode, because I can comment right on the plan and see what changed in the next version. Terminal plan mode works too, but you end up scrolling back to compare versions.

For frontend designs

- **Send a design to be inspired by.** Screenshot the Apple homepage and say "I want my website theme Apple inspired", or Airbnb, or whatever you like. Take the *style* (spacing, type, colors), not their logos or images.
- **See the design before implementing it.** Implementing means a lot of code; a mockup is a few lines of HTML and CSS. Ask for a single static HTML mockup, open it in your browser, fix the look there, then build. (The AI can't see the page itself unless you give it a browser tool, so screenshot it and paste it back.)
- **Ask for 2–3 variants** of the mockup and pick one. Cheaper than ten rounds of "make it better".
- **Put the look in the rules file:** brand color, font, light or dark, component library. Every page then comes out consistent.
- **Use a component library** the AI knows well (e.g. shadcn/ui + Tailwind). Fewer hand-made buttons, fewer weird bugs.
- **Run Impeccable on it** once a page works: "critique this page", then "polish it". Good for that last 10% before judging.
- **Say what to avoid.** "No purple gradients, no emoji icons, no glassmorphism" kills the generic AI look fast.
- **Fix with screenshots.** Something looks off? Screenshot it, circle it, paste it: "this gap is too big". (In Drawa, paste it on the canvas, draw on it, and @ it in your message.) Words like "make it pop" don't work.
- **Check it on your phone.** Most people open links on their phone first, and AI-made layouts often break on small screens (overflowing tables, tiny buttons, fixed widths). Ask for "mobile first" up front; it is way cheaper than fixing it later.
- **One page at a time.** Get the main screen right before touching the rest.

Working as a team

- **One agent session per person, one feature per session.** Two agents editing the same file = merge pain.
- **Split by area:** frontend / backend / data / pitch. Someone owns the demo script from hour 1.
- **One branch per person,** merge often, commit after every working step.
- Keep an eye on which files each agent is touching (Drawa shows this per session). If two are in the same file, stop one.

Saving tokens (and money)

- **Fresh session per feature.** Long chats get slow, expensive and confused.
- **Point at files** (@src/app.js) instead of letting it search the whole repo.
- **Small asks.** One feature per prompt.
- Caveman + Ponytail keep both the replies and the code short.
- **Use the cheaper model for simple stuff** (renames, copy changes, styling tweaks), the big one for planning and hard bugs.

Copy-paste: rules file

Save as `CLAUDE.md` (Claude Code) or `AGENTS.md` (Codex/OpenCode) in your project root. The agent reads it every session.

```
# Project
<one sentence: X helps Y do Z>

# Stack
<e.g. Next.js, Tailwind, Supabase. Deploy on Vercel.>

# Look
<e.g. dark theme, brand color #1b7a86, font Inter, shadcn/ui>

# Rules
- Keep it simple. No new libraries without asking.
- Fake data is fine; mark it with // FAKE.
- Never put API keys in code. Use .env.local.
- Commit after every working step.
```

Copy-paste: prompts

Plan first

Plan how to add <feature>. Show me the steps and a Mermaid diagram. Don't write code yet.

Feature

Add <feature> to <file/page>. Done means: when I <action>, I see <result>. Nothing else.

Bug

When I <action>, I get <exact error> at <file:line>. I expected <result>. Find the cause, don't just hide the error.

Design mockup

Here's a screenshot I like. Make a single static HTML/CSS mockup of my <page> in this style. No logic yet.

Before judging

Explain how <file/feature> works in 5 bullet points, like I'm presenting it to judges.

Drawa cheat sheet

Key	What it does
N	New session
C / Shift+C	Next / previous session
Shift+1	Fit everything on screen
Ctrl+K	Find any window, file or command
@ in a message	Reference a file, note, sketch or plan
D	Draw mode
T / S	Sticky note / scratchpad
?	All shortcuts

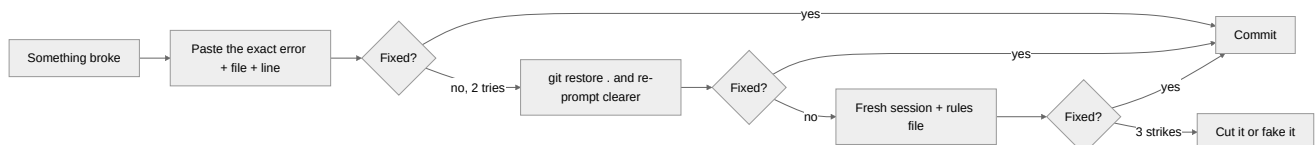
Modes (message bar): **Read only** asks before anything · **Plan only** plans, changes nothing · **Allow edits** edits without asking · **Allow everything** never asks (only for code you trust).

`drawa --net` gives you a link and QR code so your teammates can open the same canvas. Only do that on Wi-Fi you trust, since anyone with the link can run commands on your machine.

Git rescue card

```
git add -A && git commit -m "quiz page works" # after every working step
git restore . # AI broke it: undo uncommitted
git log --oneline # find the last good commit
git reset --hard <commit> # go back (loses later work!)
git switch -c my-feature # one branch per teammate
```

Stuck?



Checklists

Hour 0–1

- ☐ One-sentence pitch

- ☐ One core flow (3–5 steps)
- ☐ Boring stack picked
- ☐ Rules file written
- ☐ Architecture diagram agreed on

Last 3 hours

- ☐ Feature freeze, bug fixes only
- ☐ Deployed, demo from the live URL
- ☐ Fake data seeded so the demo always looks good
- ☐ Every diff reviewed
- ☐ `.env` not in git (`git status` before pushing)
- ☐ Demo rehearsed twice
- ☐ Backup video recorded

Pitch template

1. **Problem** (20s): who hurts and why.
2. **Live demo** (2 min): the one core flow, nothing else.
3. **How it works** (30s): show the architecture diagram.
4. **What's next** (10s): one sentence.

Know your own app. "The AI wrote it" is not an answer.

No Claude subscription?

Easiest: Drawa supports **OpenCode** out of the box, and OpenCode has free models with no key. Install it with `curl -fsSL https://opencode.ai/install | bash`, then pick **OpenCode** under **New session ▼** in Drawa. Codex works too if you have ChatGPT.

Want Claude Code itself without a subscription? Look at [OmniRoute](#), a free, open-source gateway that can route Claude Code to other model providers. Follow its own setup guide. Free providers may train on what you send, so don't use them on code you can't share.